

HENRY

HARDWARE ENABLED NEURAL RECOGNITION SYSTEM



ELEC70142 – Digital VLSI System-On-Chip (SoC) Design

CONTENTS

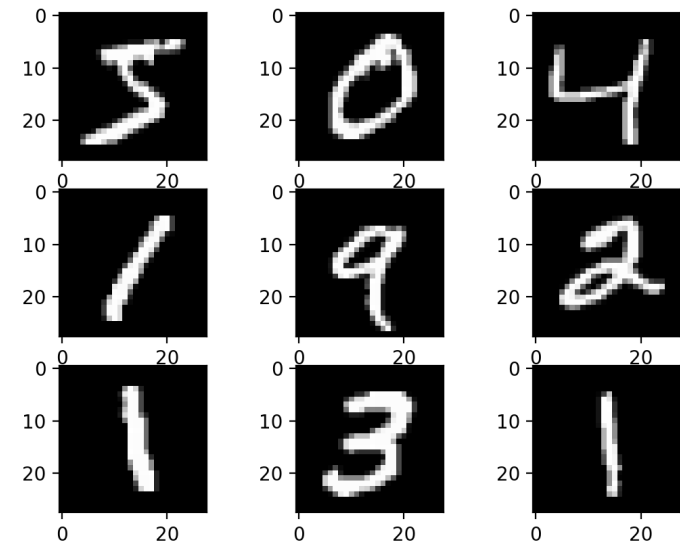
1. Our Design
2. Verification Strategy
3. Implementation
4. Testing Strategy post Tape-Out

DESIGN - OVERVIEW

A Convolutional Neural Network on a chip!

AIM:

1. Be able to classify MNIST digits using a CNN
2. Be able to change weights post-tapeout
3. Have a simple input and output interface
4. Have solid framework in place for testing post-tapeout



DESIGN – TOPLEVEL

Top-level modules

SPI

Serial interface — programs weights, loads pixels, controls inference

Neural Top

Orchestrates convolution → dense pipeline

Memory Controller

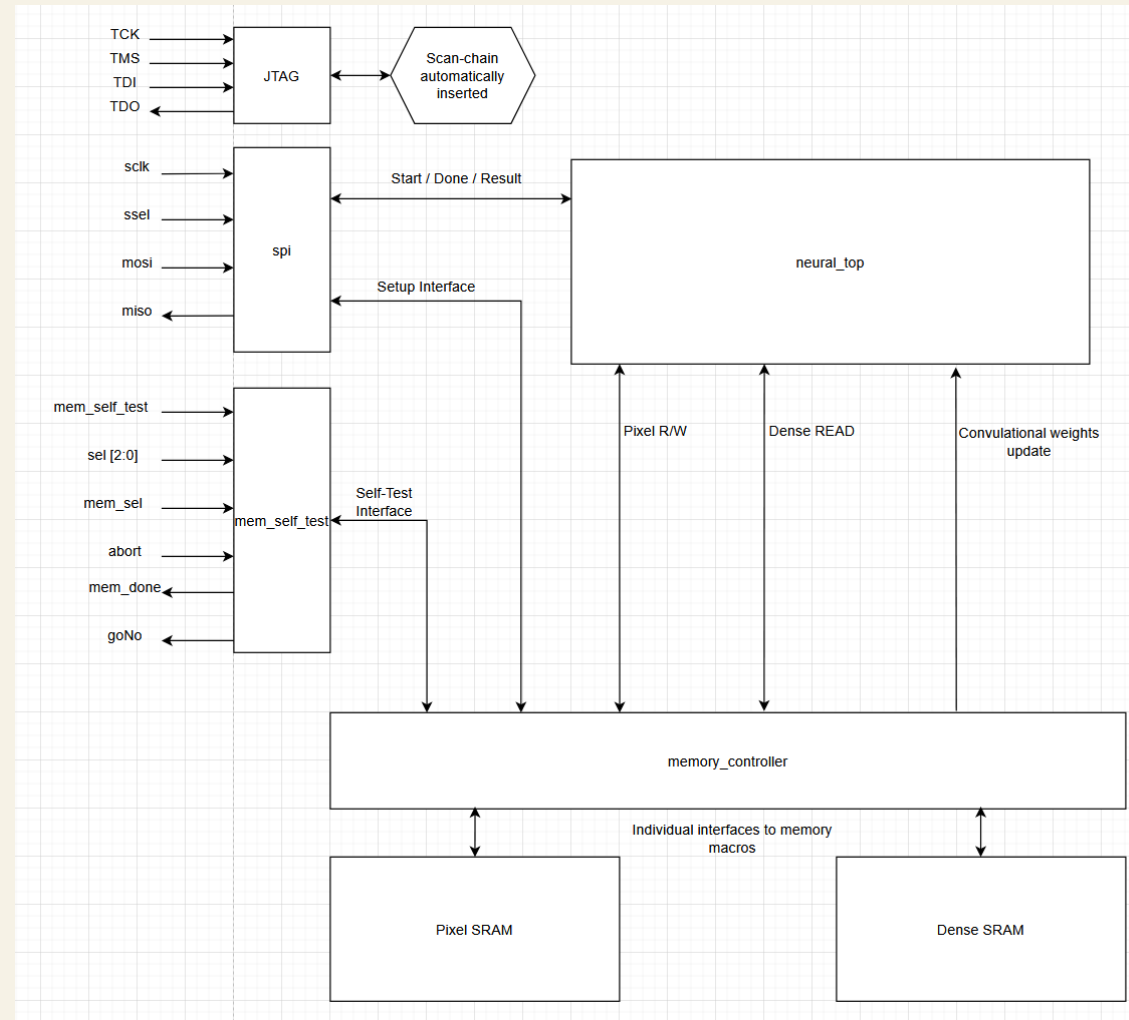
Unified byte-addressable memory map via SPI

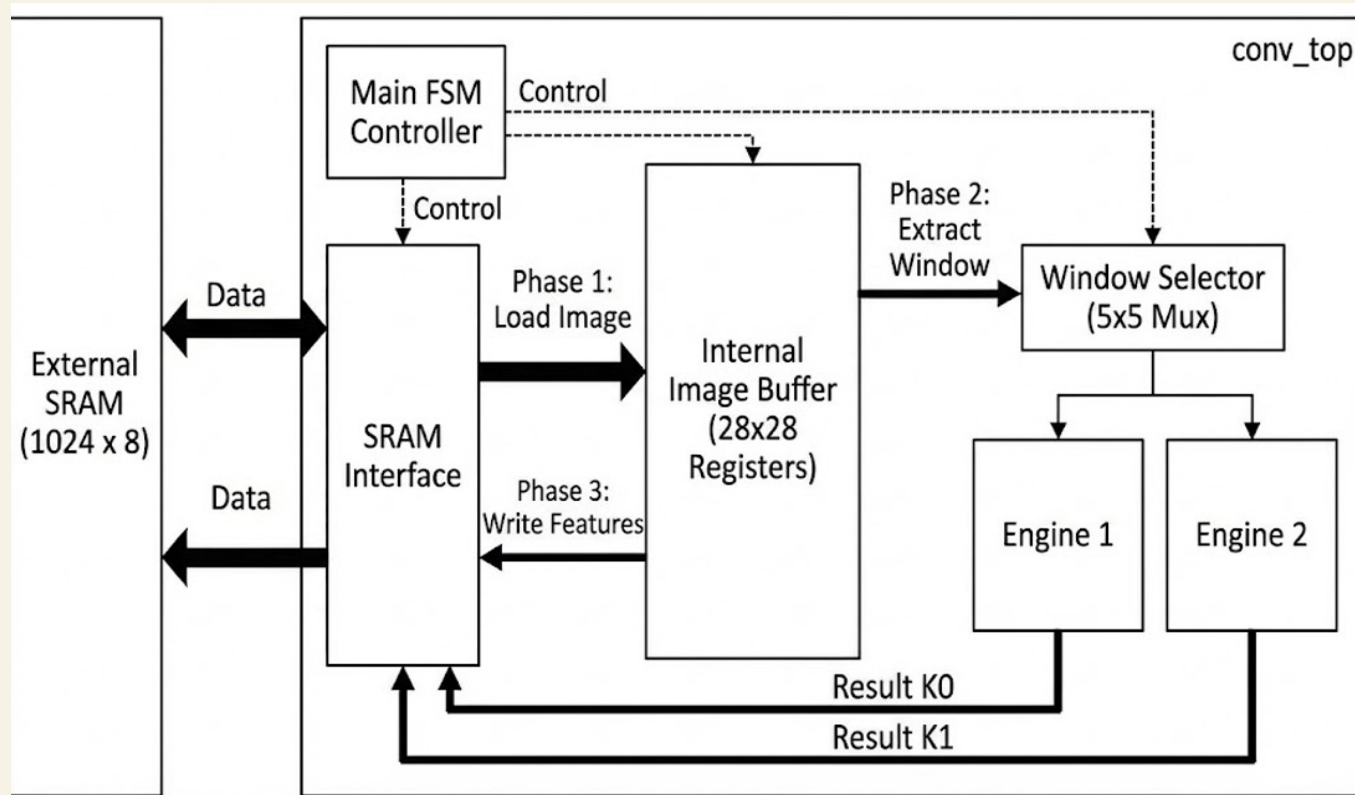
Pixel SRAM

2 000 × 8-bit (image input + conv outputs)

Dense SRAM

1 200 × 80-bit (FC weights + biases)





DESIGN - CONV ENGINE

Convolution layer

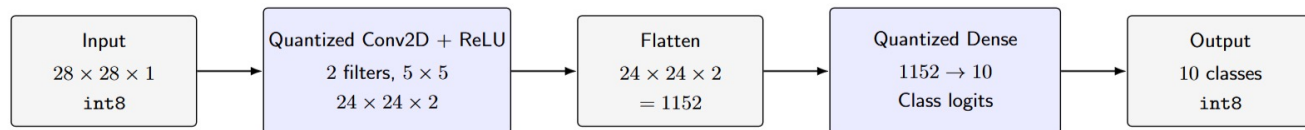
Kernel: 5×5, 2 filters, stride 1

Activation: ReLU (3-bit truncation)

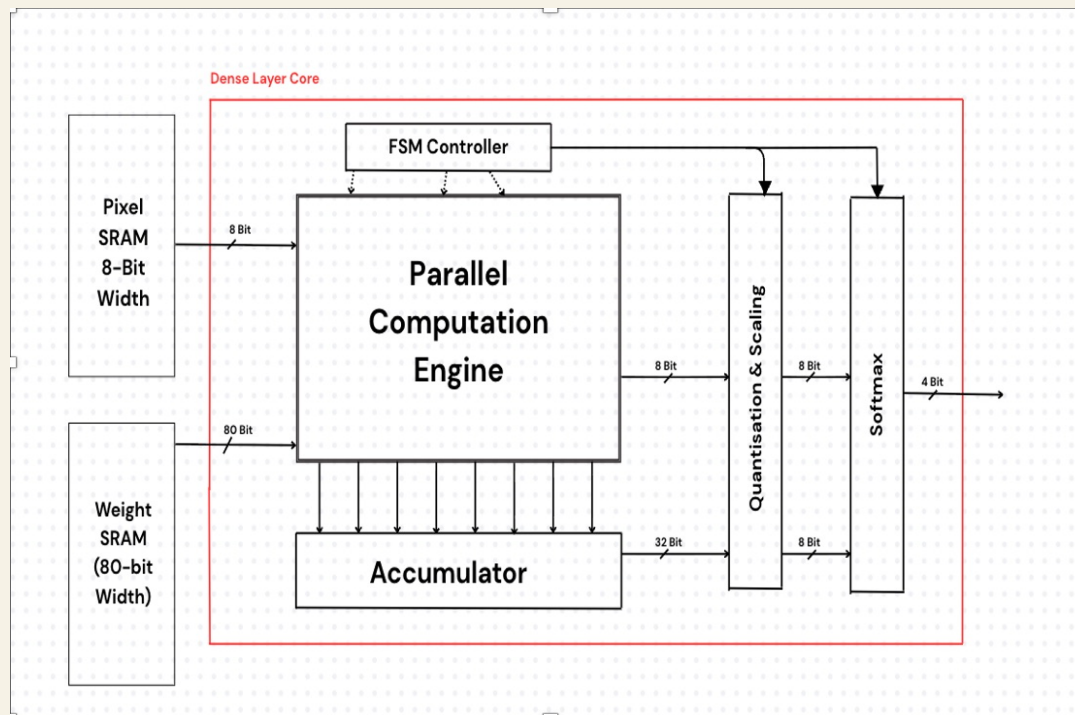
Prefetch buf: Loads 5×5 window from Pixel SRAM

MAC: INT8 accumulator + bias

Output: 2 feature maps → Pixel SRAM



DESIGN – DENSE LAYER



Single Fully Connected Layer

1152 $\rightarrow$ 10 output classes
(input: $2 \times 24 \times 24$ conv feature maps)

Weights: INT8, stored in Dense SRAM

SRAM: 80-bit wide for parallel MAC

Result: ArgMax on dense output $\rightarrow$ digit 0–9

DESIGN – NEURAL TOP

Sequencer between conv and dense engines

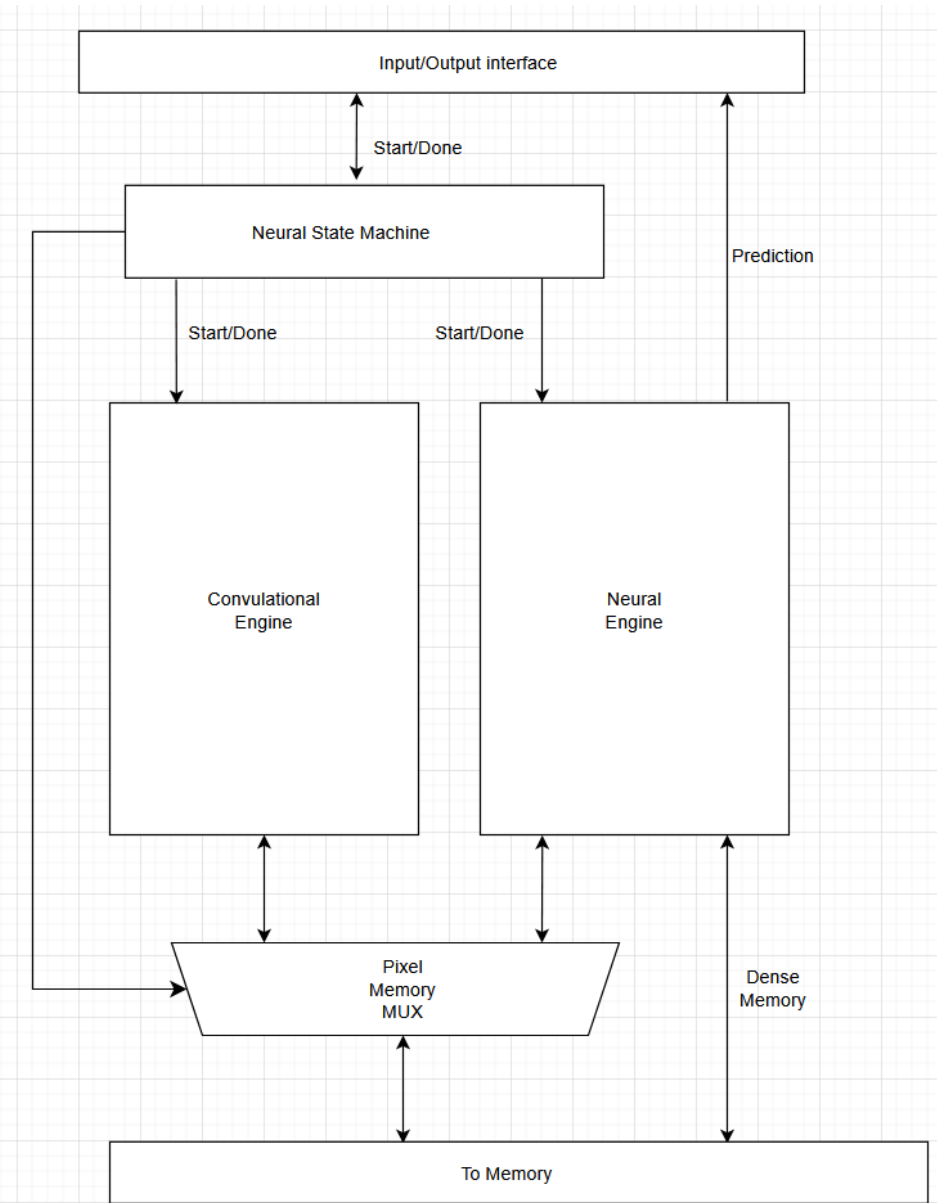
FSM: IDLE → CONV → NEURAL → DONE

- Strictly sequential — dense needs conv's output

Pixel Memory mux

- Conv owns it during CONV
- Dense owns it during NEURAL
- Parked safely in IDLE / DONE
- Ensures no bus contention

Single start pulse in → 4-bit prediction + done out



DESIGN – MEMORY

Design Choice: 2 SRAMS

- Pixel Memory (8 bit word, 2000 words)
 - For storing image and convolutional outputs
- Convolution Weights and Biases in Registers
 - Small size
- Dense Memory (80 bit word, 1200 words)
 - For storing dense weights and biases

DESIGN – MEMORY CONTROLLER

Design Choice: unified memory map

- Expose a single, unified, byte-addressable memory interface
- Accessible by SPI: good for testing.

Memory Map

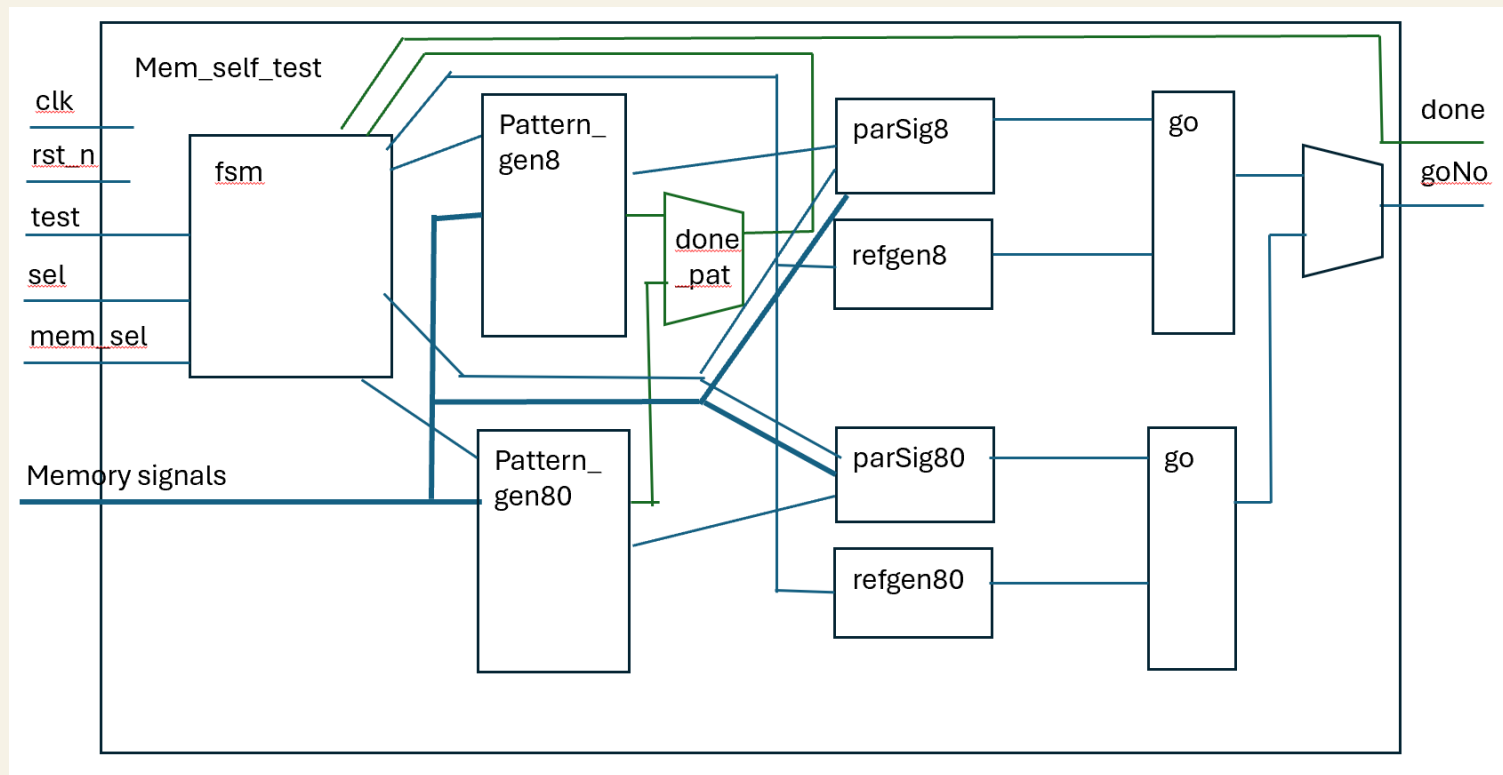
Region Name	Start Address	End Address	Size (Bytes)	Description
Pixel Input	0x0000	0x030F	784	28x28 Raw Image
Conv Out 0	0x0310	0x054F	576	24x24 Feature Map
Conv Out 1	0x0550	0x078F	576	24x24 Feature Map
Unused	0x0790	0x077F	112	
Conv Registers	0x0800	0x083F	60	Weights and Biases Register Map (WRITE ONLY)
Unused	0x0840	0x0FFF	1984	
Dense Start	0x1000	0x583F	18496	Start of Aligned Dense Region

DESIGN – MEMORY CONTROLLER

Design Choice: unified memory map

- Memory mapped control and I/O
- Start Register (0xFFFF)
- Result Register (0xFF00)
- Status Register (0xFF04)

DESIGN – MEMORY SELF TEST



DESIGN – MEMORY SELF TEST

- Memory self test on dense memory and pixel memory
- Available test patterns:

Pattern	Description	Faults covered
All-Zeros	Write 0s to all addresses, read back	stuck-at-1
All-Ones	Write 1s to all addresses, read back	stuck-at-0
Checkerboard	Write alternating 1s and 0s to all addresses, read back	adjacent cell coupling, stuck-at
Walking 1/0	Move 1/0 through all memory cells, read back	address line decoder, data line
Galloping	Move 1/0 through all memory cells, read test cell and other cells alternatively	address line decoder, data line, transition, stuck-at, coupling
Random	Write pseudo-random binary sequence to all addresses, read back	pattern sensitivity

DESIGN – MEMORY SELF TEST

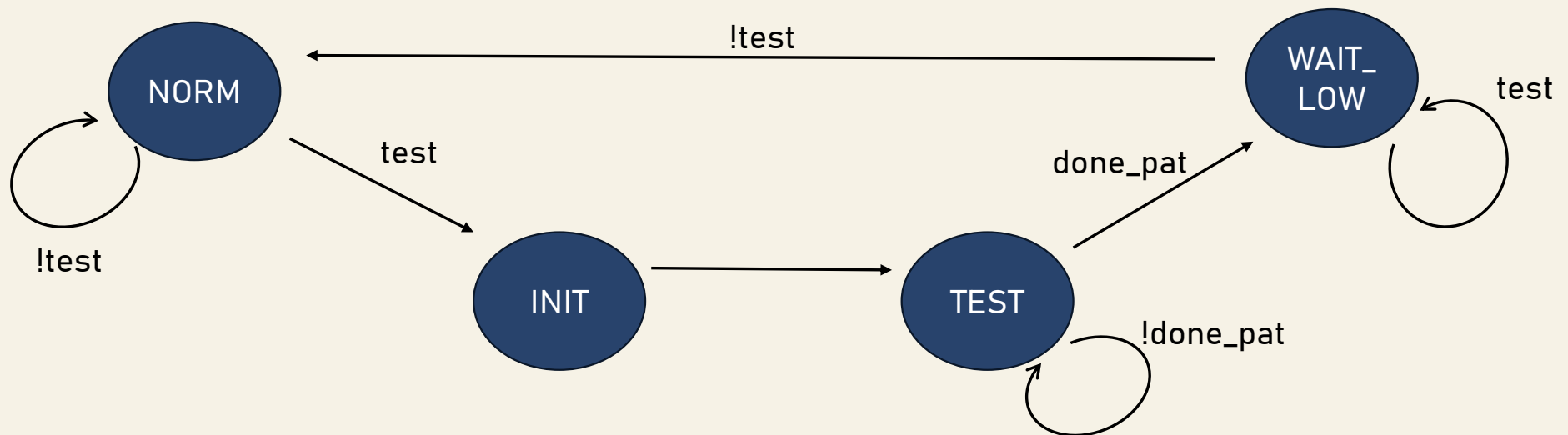
Interface

Signal	Description
test	Activate self test
sel	Test pattern
mem_sel	1: pixel memory 0: dense memory
goNo	Indicate test pass
done	Indicate test complete

DESIGN – MEMORY SELF TEST

Operation

- NORM: normal operation, SRAM memory accessed by dense neurons and convolutional layers
- INIT: initialize test pattern generation, reset signature analysers
- TEST: selected pattern iterate through memory addresses, memory reads/writes, signature generation
- WAIT_LOW: signature comparison with stored signature, hold test pass/fail output until next test rising edge



DESIGN – MEMORY SELF TEST

Implementation

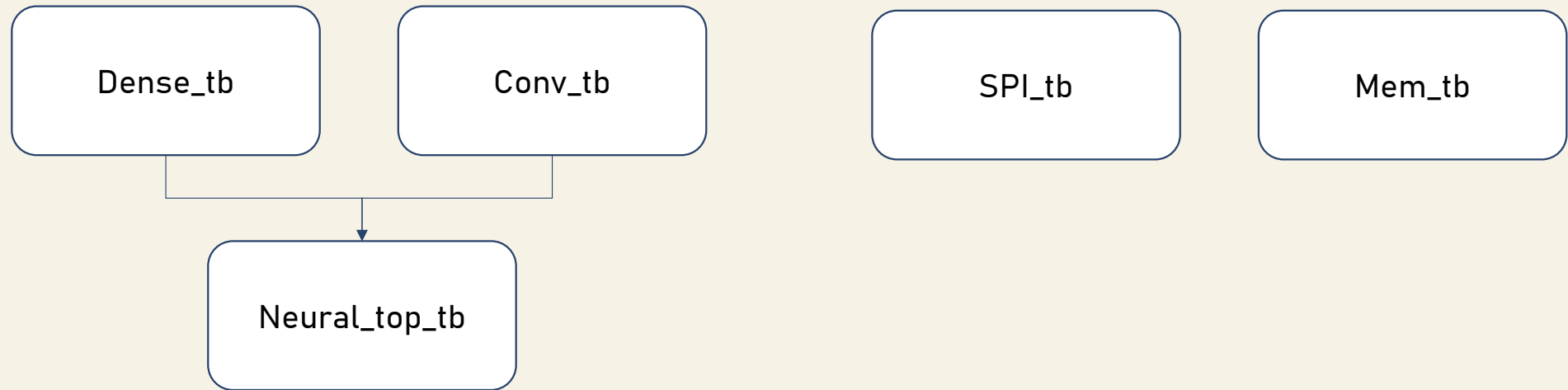
- **Finite state machine:** memory management
- **Counters:** address sequence, read/write sequence
- **Random number generators:** pseudo-random data
- **Signature analyzers:** data compression into signature

VERIFICATION - STRATEGY



- Detailed testing at all design stages
→ Catches different bugs
- Toplevel testbench reused for post-synthesis and post-layout simulations
→ Ensures correctness with RTL.
- How? CocoTB + IcarusVerilog

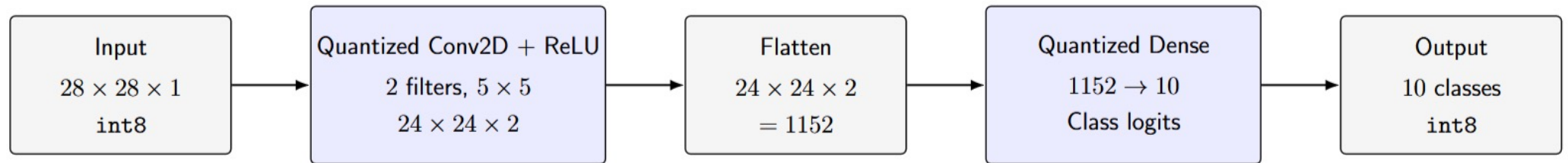
VERIFICATION – UNIT LEVEL



AIM: To test each unit independently to ensure smooth integration.

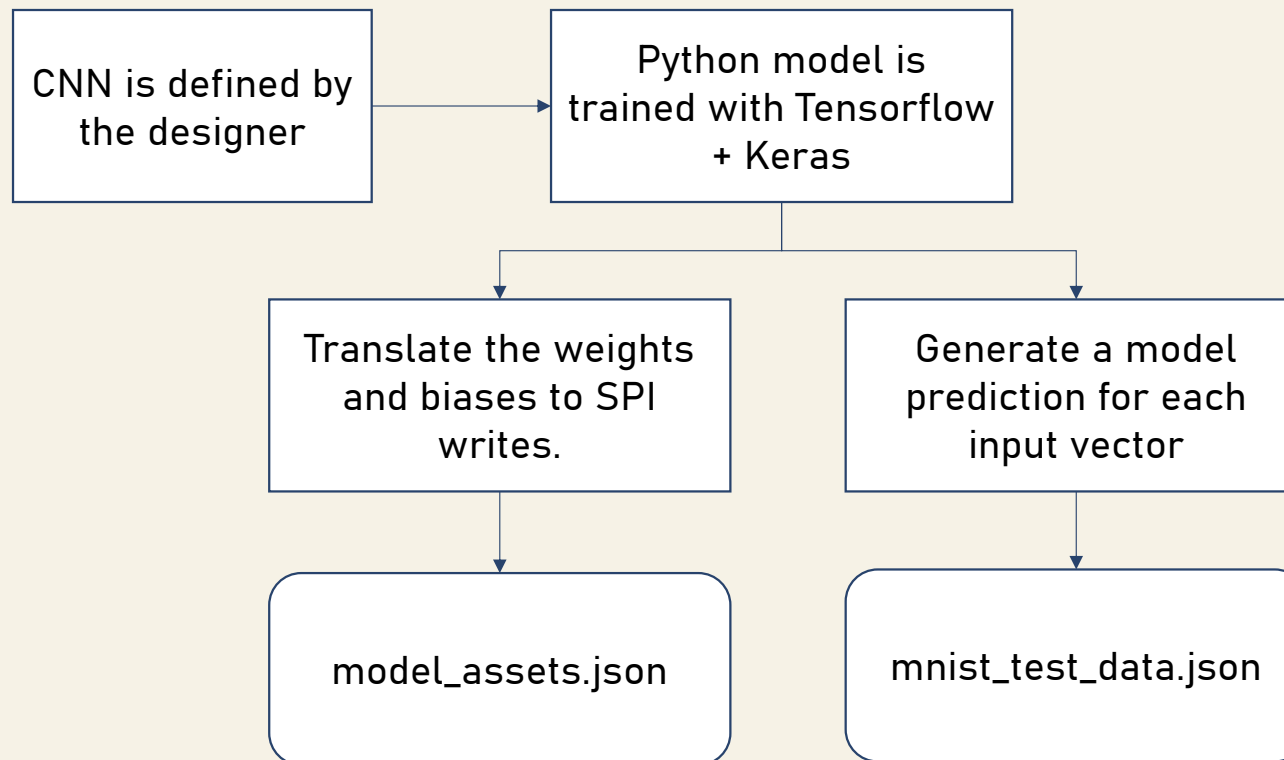
Vector compared against detailed models for each unit.

VERIFICATION – MODEL DEF.

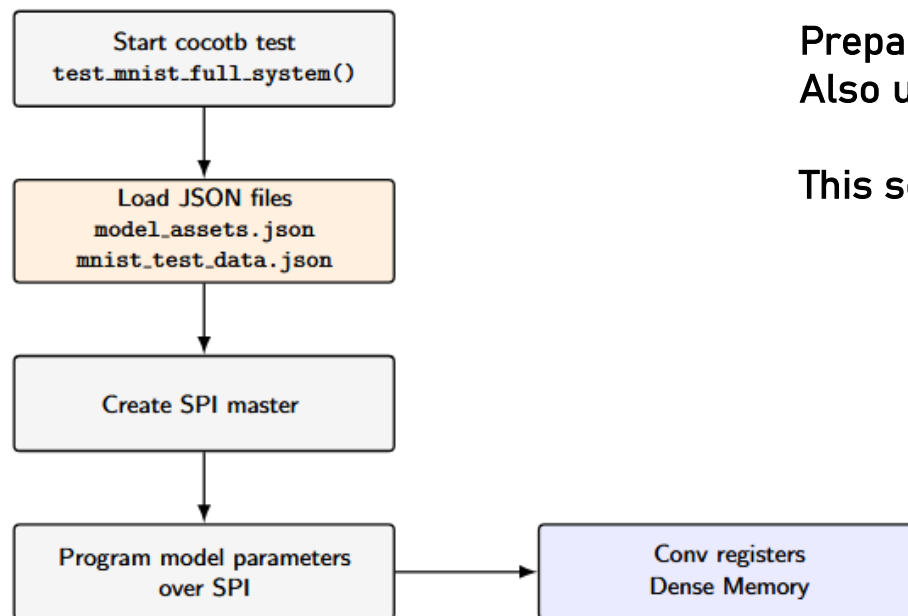


- Baseline CNN model “nano” LENET

VERIFICATION – MODEL STRAT.



VERIFICATION – TOPLEVEL TB



Prepare the DUT for testing.

Also useful to read-back the values from memory.

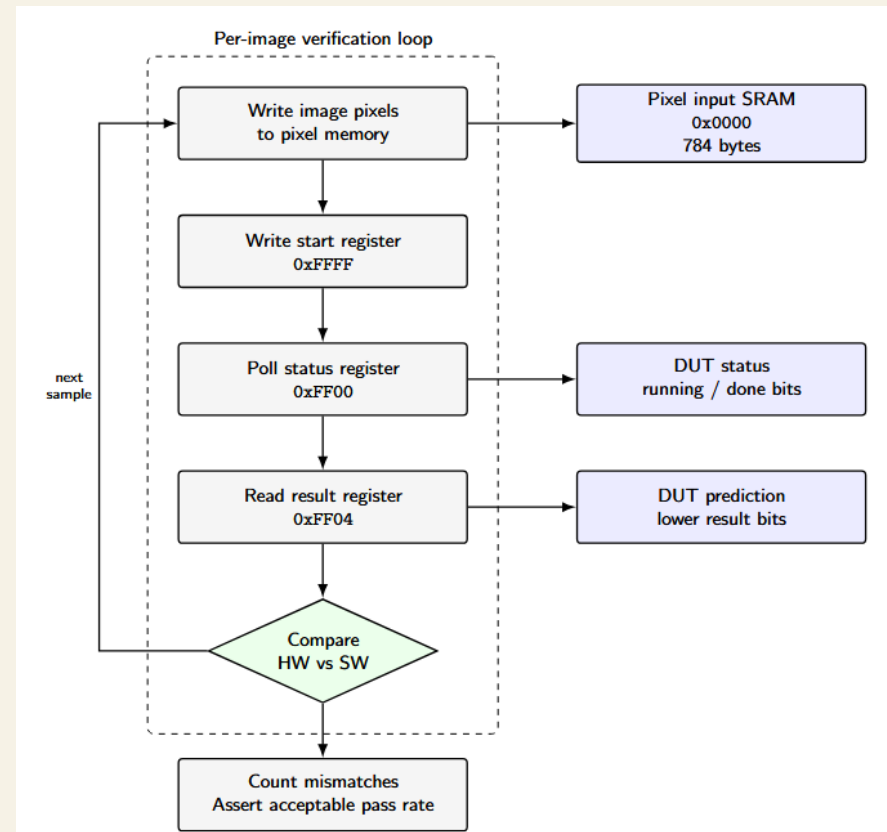
This serves as checks on the SPI + Memory Read and Write

VERIFICATION – TOPLEVEL TB II

Tests predictions on sustained workload.

Acceptable pass rate:
5 mismatches during early verification

0 mismatches during late verification



VERIFICATION – POST SYNTHESIS POST-LAYOUT

- Verify that synthesis did not optimize away our design.
- Synthesis did in fact optimize away our design.
- Because of a typo inferring a latch, SPI module was deleted.
- Separate Makefile, wrapper around the netlist.

VERIFICATION - RESULTS

Pass at Unit-Level, Top-Level, Gate-Level and Layout.

But could we have done better?

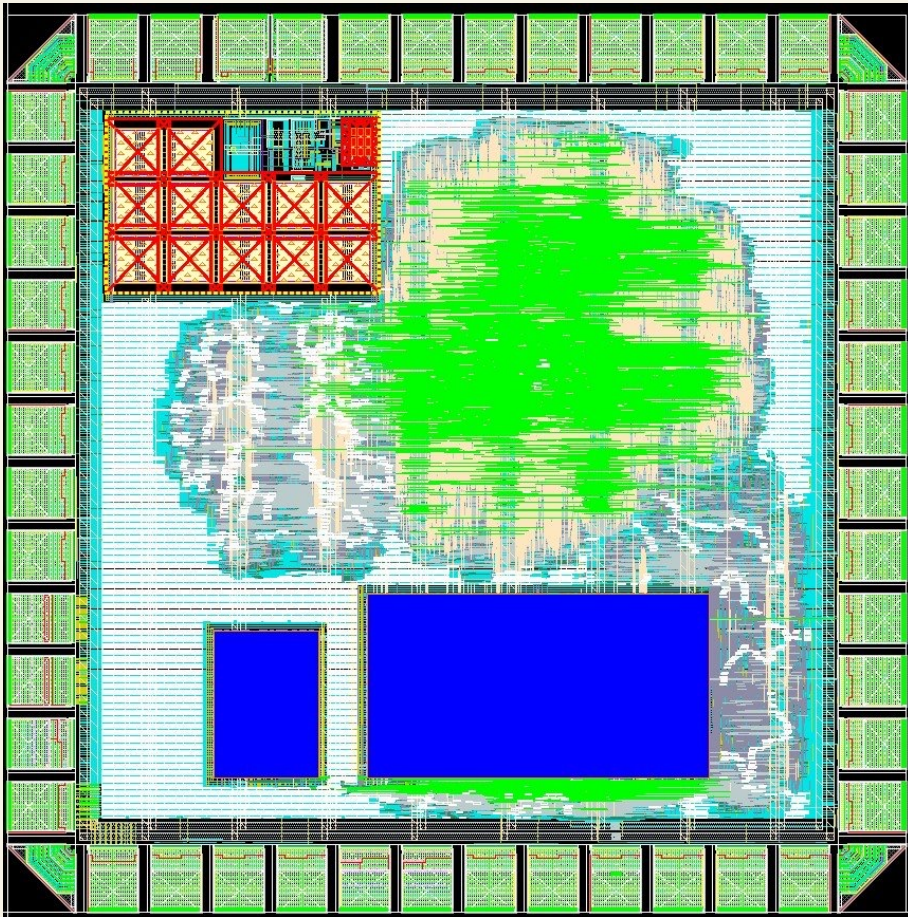
- Coverage?
- Formal verification strategy?
- > Good enough for us.

IMPLEMENTATION - FLOW



- **RTL:** System Verilog
- **Verification:** Cocotb + Icarus Verilog
- **Synthesis:** Cadence Genus, DFT scan chain insertion
- **Place & Route:** Cadence Innovus
- **Signoff:** DRC, LVS, Cadence XCellium
- **Fabrication:** TSMC 65nm process

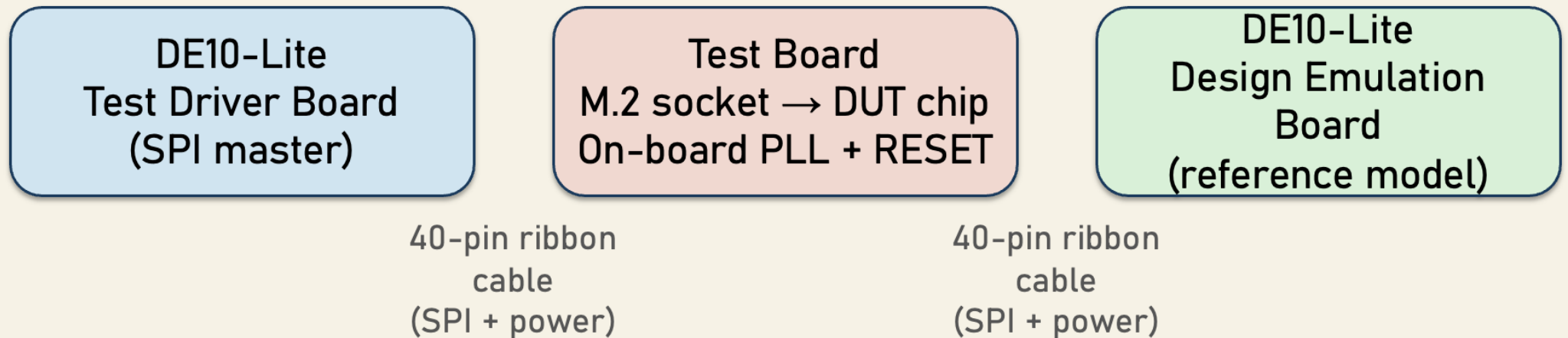
IMPLEMENTATION – GDSII



- **GPIO Padframe:** SPI, debug signals, PLL control signals, external clock in/out, power
- **On-chip SRAM macros:** pixel memory (2000x8), dense memory (1200x80)
- **PLL macro:** internal system clock
- **Digital standard cell logic:** SPI slave, memory controller, convolutional layers, dense neuron layers, memory self test, PLL check logic
- **Improvements:** refine memory macro placement, more GPIO pins

TESTING – HARDWARE SETUP

Three-board setup: Test Driver FPGA and Design Emulation FPGA flank a custom Test Board that holds the chip in an M.2 socket.



Host PC connects to Test Driver via USB — Python scripts automate SPI transactions and log results.

TESTING – 6-PHASE PLAN

PHASE 1 POWER / IO

Supply ramp-up
Current limits
Pad continuity
I/O voltage levels

PHASE 2 SPI

Register R/W
Memory map
Mode control
Readback check

PHASE 3 JTAG

TAP controller
Boundary scan
Scan chain shift
Stuck-at coverage

PHASE 4 FSM

Start command
State transit.
Status register
Done/error flags

PHASE 5 DATAPATH

Load known img
Run inference
Check conv outs
Dense result

PHASE 6 MNIST E2E

100 test images
HW vs TFLite
>90% agreement
Accuracy log

TESTING – PHASE 1 & 2

PHASE 1: ELECTRICAL

- ✓ Apply VDD; verify supply current within spec
- ✓ Continuity test on all bond pads
- ✓ Measure high/low I/O voltage levels
- ✓ Confirm clock signal present (25 MHz)
- ✓ Check reset behaviour (active-low RST_N)

PHASE 2: SPI INTERFACE

- ✓ Write/read status register (addr 0xFF00)
- ✓ Write pixel memory (addr 0x0000, 784 bytes)
- ✓ Write dense weights (addr 0x8000)
- ✓ Verify readback matches written values
- ✓ Test mode-select bits via SPI control word

TESTING – PHASE 3: JTAG

TAP CONTROLLER

- ✓ Verify TDI / TDO / TMS / TCK / TRST pins active
- ✓ Step TAP state machine via TMS sequences
- ✓ BYPASS instruction: TDI feeds directly to TDO
- ✓ IDCODE instruction: read chip identification register
- ✓ Confirm TRST resets TAP to Test-Logic-Reset state

SCAN CHAIN DFT

- ✓ Shift walking-1 pattern through all flip-flops
- ✓ Compare scan-out vs expected (stuck-at-0 / stuck-at-1)
- ✓ Apply transition fault vectors (0→1 and 1→0)
- ✓ Boundary scan (EXTEST): verify all pad connections
- ✓ Trigger Dense SRAM BIST via JTAG user instruction

TESTING – PHASE 4, 5 & 6

PHASE 4: FSM & CONTROL

- ✓ Send start command via SPI
- ✓ Monitor state transitions
- ✓ Poll running/done bits (0xFF00)
- ✓ Verify error/abort flags

PHASE 5: DATAPATH

- ✓ Load known MNIST image via SPI
- ✓ Run inference; read conv outputs
- ✓ Compare HW result vs TFLite model

PHASE 6: MNIST END-TO-END

- ✓ Stream 100 MNIST test images
- ✓ HW prediction vs TFLite software
- ✓ Target: $\geq 90\%$ agreement
- ✓ Log all mismatches for analysis

LESSONS LEARNT

Agreement between designers and verifiers matter.

Mismatch between what the model does (or what I thought the model did)

And how the design was implemented in RTL.

Architecture decisions compound

Our conv engine started as a streaming pipeline; a mid-project pivot to a prefetch-buffer architecture (to match shared SRAM) cost us Conv BIST time.

LESSONS LEARNT II

DFT must be designed in from day one

Conv BIST did not make tapeout because it was added too late. Self-test logic belongs in the initial RTL architecture.

Design for observability before tapeout

More focus on observability. We could have used more GPIO pins to observe more parts of the design for example.

THANK YOU!
QUESTIONS?